
Classes

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya)

Introduction to Object-Oriented Programming

- Information about the course
- First glance at Java
- Programming paradigms
- Object-oriented concepts
- Principles of the object-oriented paradigm

Principles of the object-oriented paradigm

- Abstraction
 - Expose essential details
- Encapsulation
 - Combine data and operations in an entity
- Inheritance
 - Derive new types
- Polymorphism
 - Behavior depends on the context

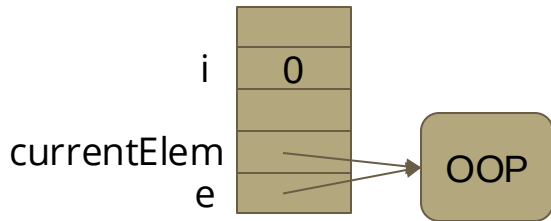
```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (int i=0; i < this.elems.length; i++) {  
            String currentElem = this.elems[i];  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

Data Types

Values have types

primitive types - e.g., int, double, bool, char, ...

reference types - e.g., String, null, Catalog.



```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (int i=0; i < this.elems.length; i++) {  
            final String currentElem = this.elems[i];  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

Identifiers must have a Java letter or \$ or _ as first character - e.g., elems

final indicates that currentElem is a constant

Quiz 1 - Data types and Operators

Please provide your input using [this form](#)

== and reference types / String

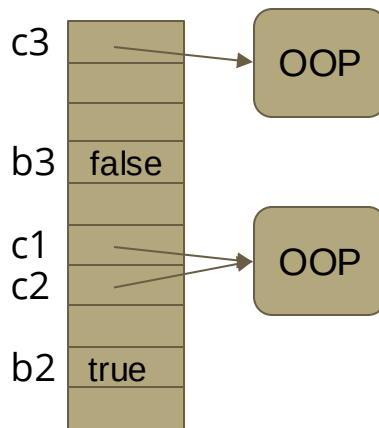
String c1 = "OOP";

String c2 = "OOP";

boolean b2 = (c1 == c2);

String c3 = **new** String("OOP");

boolean b3 = (c1 == c3);



```
jshell> String c1 = "OOP";  
c1 ==> "OOP"  
  
jshell> String c2 = "OOP";  
c2 ==> "OOP"  
  
jshell> boolean b2 = (c1 == c2);  
b2 ==> true  
  
jshell> String c3 = new String("OOP");  
c3 ==> "OOP"  
  
jshell> boolean b3 = (c1 == c3);  
b3 ==> false
```

Evaluates whether both operands **refer** to the same object

What are the values of b2 and b3 after executing these commands?

Due to the **new** operator a new object was created

Data types and operators

- `==` requires both operands to have the same (or compatible) type.
- Some types can be **coerced** (implicitly converted) to other types, e.g. a short or byte can be coerced to int.
- For reference types, `==` simply compares the references.
- Strings are **immutable** (cannot be changed).

Statements

types are specified when
declaring a variable
(Java is **strongly typed**)

enhanced **for** is used to
iterate over elements of
arrays and collections

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (String currentElem : this.elems) {  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

Classes

- Fundamental part of object-oriented programs.
- Templates used to create objects.
- Define the **properties** (fields) and **behavior** (methods and constructors).
- **Fields** and **methods** and **constructors** are called **members** of the class.
- Classes have at least one constructor (used to initialize objects).
- Constructors have the same name as the class.
- Classes define new reference types.

Classes

Optional **modifiers**

Optional **initialization** of fields in their declaration

Default initialization:
number: 0
boolean: false
reference type: **null**

this is a final reference to the **current instance** of the class

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (String currentElem : this.elems) {  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

elems and size are **instance fields**, i.e., each object has its own copy of these fields

count is a **class field**, i.e., there is only one copy
Declared using the **static** modifier

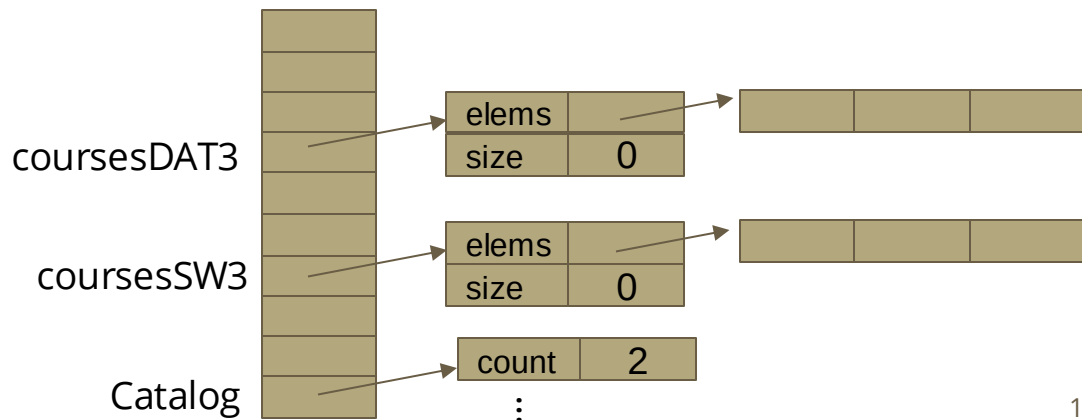
A class with two instance variables, one class variable, one constructor, and two methods

Instances

```
Catalog coursesSW3 = new Catalog(3);  
Catalog coursesDAT3 = new Catalog(3);
```

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    ...  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    ...  
}
```

- The **new** operator **creates** an object (instance of the class).
- The constructor **initializes** the instance.
- **Memory** is allocated on heap.
- A **reference** to the instance is returned.



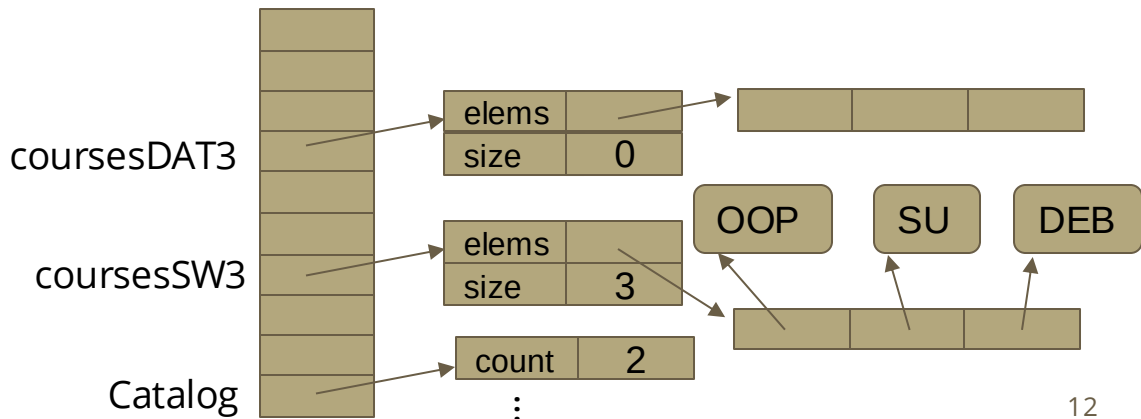
Instances

```
Catalog coursesSW3 = new Catalog(3);  
Catalog coursesDAT3 = new Catalog(3);  
coursesSW3.addElem("OOP");  
coursesSW3.addElem("SU");  
coursesSW3.addElem("DEB");
```

```
public class Catalog {  
    private String[] elems = null;  
    ...  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    ...  
}
```

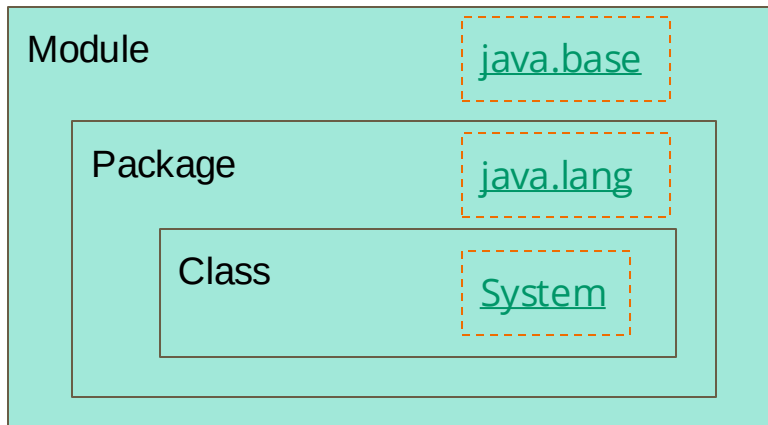
- **Dot notation** used to access the class members.
- Visibility modifier **private** does not (in general) allow for the following command:

```
coursesSW3.elems[0] = "OOP";
```



Packages and modules

- Classes can be grouped into **packages** of related classes
- Packages can be grouped into **modules** of related packages



Packages and modules can have qualified names (i.e., including '.')

```
import java.lang.System;

public class Catalog {
    ...
    public static void main(String[] args) {
        ...
        System.out.println(result);
    }
}
```

If no module is specified, the class is member of the *unnamed module*

No ambiguity is allowed

Import-on-demand (`*` at the end) allows to potentially import all the classes from a package, e.g.
`import dk.aau.adt.*;`

Accessing classes from other packages

```
package dk.aau.adt;

public class Catalog {
    private String[] elems = null;
    private int size = 0;
    private static int count = 0;
    public Catalog(int maxSize) {
        this.elems = new String[maxSize];
        count++;
    }
    public void addElem(String e) {
        this.elems[size] = e;
        this.size++;
    }
    ...
}
```

Types in the **java.lang** package are imported automatically, e.g., `String`, `System`

```
package dk.aau.tests;

import dk.aau.adt.Catalog; // single-type import

class CatalogTest {
    public static void main(String args[]) {
        Catalog coursesSW3 = new Catalog(3);
        ...
    }
}
```

Catalog must be **public** to be used from other package

```
package dk.aau.tests;

class CatalogTest2 {
    public static void main(String args[]) {
        dk.aau.adt.Catalog coursesSW3 = ...
    }
}
```

Use the fully **qualified name**

No ambiguity is allowed

java.base exports several packages, e.g., **java.lang**, **java.io**

Accessing classes from other modules

Module **aau.adt**

package dk.aau.adt;

aau.adt/dk/aau/adt/Catalog.java

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    ...  
}
```

qualified export

```
module aau.adt {  
    exports dk.aau.adt to aau.tests;  
}
```

aau.adt/module-info.java

Module **aau.tests**

package dk.aau.tests;

aau.tests/dk/aau/tests/CatalogTest.java

```
import dk.aau.adt.Catalog;  
  
class CatalogTest {  
    public static void main(String args[]) {  
        Catalog coursesSW3 = new Catalog(3);  
        ...  
    }  
}
```

Catalog must be **public** to be used from other package

```
module aau.tests {  
    requires aau.adt;  
}
```

aau.tests/module-info.java

modules export **packages**
modules require **modules**

No ambiguity is allowed

Accessing classes from other modules

Module **aau.adt**

```
package dk.aau.adt;
```

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    ...  
}
```

aau.adt/dk/aau/adt/Catalog.java

Module **aau.tests**

```
package dk.aau.tests;
```

```
import dk.aau.adt.Catalog;  
  
class CatalogTest {  
    public static void main(String args[]) {  
        Catalog coursesSW3 = new Catalog(3);  
        ...  
    }  
}
```

aau.tests/dk/aau/tests/CatalogTest.java

Catalog must be **public** to be used from other package

```
module aau.tests {  
    requires aau.adt;  
}
```

aau.tests/module-info.java

unqualified export

```
module aau.adt {  
    exports dk.aau.adt;
```

aau.adt/module-info.java

Quiz 2 - Classes and Instances

Please provide your input using [this form](#)

Methods

- Named block of code that defines the behavior of **the objects** of that class (instance or non-static method) or **the class** (class or **static** methods).
- Can be **overloaded**, i.e., there can be more than one, but with a different signature.

Two instance methods:

return type void, name addElem, arities one and two, signatures (String) and (String int),

One **class method** with return int, arity zero, and name getCount.

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public void addElem(String e, int k) {  
        for (int i=1; i<=k; i++) {  
            this.elems[size] = e+i;  
            this.size++;  
        }  
    }  
    public static int getCount() {  
        return Catalog.count;  
    }  
}
```

Signature: name + sequence of parameter types.
Every method and constructor has a **unique** signature.

Constructors

- Have the same name as the class.
- Do not have a return type.
- Can be **overloaded**, i.e., there can be more than one, but with a different signature.
- If no constructors are declared, then one with arity zero is implicitly declared (default).
 - With the same access level as the class
- A constructor can call another one using the keyword **this**.
 - It must be the **first** statement in the constructor.

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        elems = new String[maxSize];  
        count++;  
    }  
    public Catalog(String[] elems) {  
        this(elems.length);  
        for (String currentElem : elems) {  
            elems[size] = currentElem;  
            size++;  
        }  
    }  
    public int getSize() {  
        return size;  
    }  
    public String getFirst() {  
        return elems[0];  
    }  
}
```

Initialization Blocks

- Block of code outside any methods or constructors.
- Are executed when an instance is created (before constructors).
- But if they are static, they are executed only once and **cannot** refer to instance variables or **this**.
- They **cannot** have return statements.

```
public class Catalog {  
    private String[] elems;  
    private int size;  
    private static int count;  
    {  
        this.size = 0;  
    }  
    static {  
        count = 0;  
    }  
    ...  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public static int getCount() {  
        return Catalog.count;  
    }  
    public int getSize() {  
        return size;  
    }  
    public String getFirst() {  
        return elems[0];  
    }  
}
```

Parameters

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    ...  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
}
```

```
Catalog coursesSW3 = new Catalog(3);  
String course1="OOP";  
coursesSW3.addElem(course1);  
coursesSW3.addElem("SU");
```

Input is passed to the method using its parameters, e.g., **e** is the **formal parameter** of addElem

course1 and **"SU"** are **actual parameters**

Scope and hiding

- **Local variables** must be initialized before it can be used, their scope is the code block where they are defined.
- **Instance and class variables** have the entire class as their scope (where the variable can be referred to with its simple name). `this.` can be omitted in lines 6, 10, 11, 15.
- **Formal parameters** of a method are treated as local variables.

```
1. public class Catalog {
2.     private String[] elems = null;
3.     private int size = 0;
4.     private static int count = 0;
5.     public Catalog(int maxSize) {
6.         this.elems = new String[maxSize];
7.         count++;
8.     }
9.     public void addElem(String e) {
10.        this.elems[size] = e;
11.        this.size++;
12.    }
13.    public String find(String e) {
14.        String res;
15.        for (String currentElem : this.elems) {
16.            if (currentElem==e) {
17.                res = currentElem;
18.            }
19.        }
20.        return res;
21.    }
22. }
```

Scope and hiding

- Local variables **hide** instance or class variables

```
String[] strings = { "OOP", "SU"};  
Catalog c = new Catalog(strings);
```

What is the size of c?

What is the first element of c?

```
1. public class Catalog {  
2.     private String[] elems = null;  
3.     private int size = 0;  
4.     private static int count = 0;  
5.     public Catalog(int maxSize) {  
6.         elems = new String[maxSize];  
7.         count++;  
8.     }  
9.     public Catalog(String[] elems) {  
10.        this(elems.length);  
11.        for (String currentElem : elems) {  
12.            this.elems[size] = currentElem;  
13.            size++;  
14.        }  
15.    }  
16.    public int getSize() {  
17.        return size;  
18.    }  
19.    public String getFirst() {  
20.        return elems[0];  
21.    }  
22. }
```

Quiz 3 - Scope and hiding

Please provide your input using [this form](#)

Class and instance methods

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public static int getCount() {  
        return Catalog.count;  
    }  
}
```

getCount cannot use **instance variables**

addElem cannot be invoked
before the command in line 2

1. int c = **Catalog**.getCount();
2. Catalog coursesSW3 = new Catalog(3);
3. String course1="OOP";
4. **coursesSW3**.addElem(course1);
5. **coursesSW3**.addElem("SU");

getCount() can be invoked before any
instance exists.

Instance methods (e.g., addElem) **need an
instance as context.**

A class method **has as context the class.**

Quiz 4 - Methods and Fields

Please provide your input using [this form](#)

main

- It is a class/**static** method
- **public** makes it accessible from any context where the class is accessible.
- No return value (void).
- One parameter of type `String[]`.
- Only classes with a **main** method can be used as entry point for Java applications (run).

```
java --module-path mod -module aau.tests/dk.aau.tests.CatalogTest
```

Module **aau.tests**

```
package dk.aau.tests;
import dk.aau.adt.Catalog;
```

aau.tests/dk/aau/tests/CatalogTest.java

```
class CatalogTest {
    public static void main(String args[]) {
        Catalog coursesSW3 = new Catalog(3);
        ...
    }
}
```

```
module aau.tests {
    requires aau.adt;
}
```

aau.tests/module-info.java

```
module aau.adt {
    exports dk.aau.adt;
}
```

aau.adt/module-info.java

Compiled modules **aau.adt** and **aau.tests** available in folder **mod**

Accessing class members

- Access level modifiers:
 - **public**: accessible everywhere the class is accessible
 - **private**: accessible only within the body of the class
 - **protected**: from the same package or a *descendant* of the class
 - default or package-level (no modifier)
 - Typical use: helper classes

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public static int getCount() {  
        return Catalog.count;  
    }  
}
```

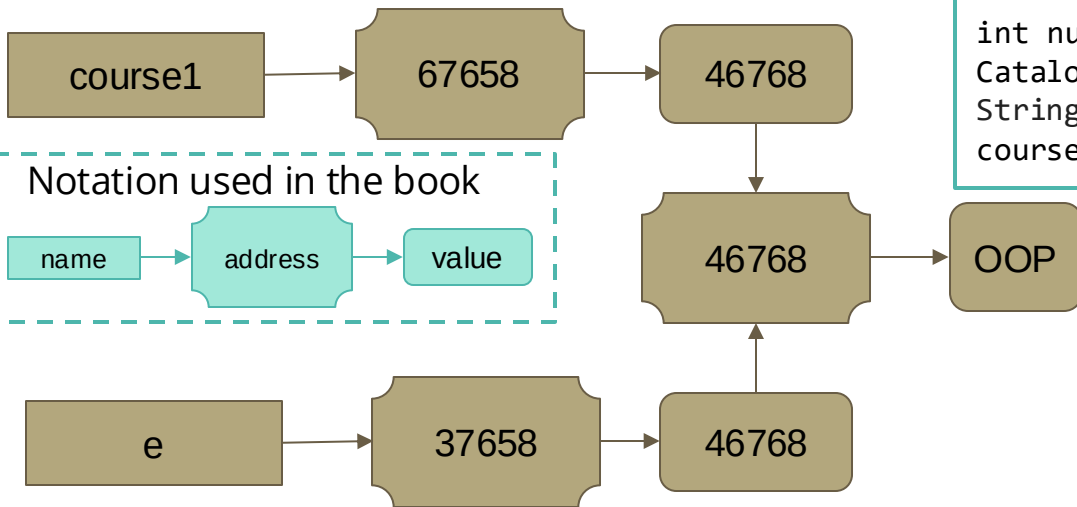
All instance variables should be declared as **private** (data hiding)
public methods can be used to access and manipulate private
variables in a **controlled way**

Parameter-Passing

Pass **by value** for primitive types

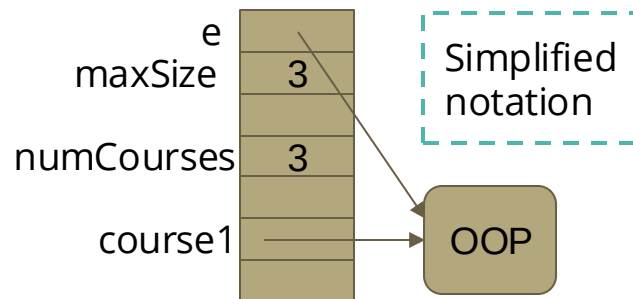


Pass **by reference value** for reference types



```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
}
```

```
int numCourses = 3;  
Catalog coursesSW3 = new Catalog(numCourses);  
String course1 = "OOP";  
coursesSW3.addElem(course1);
```

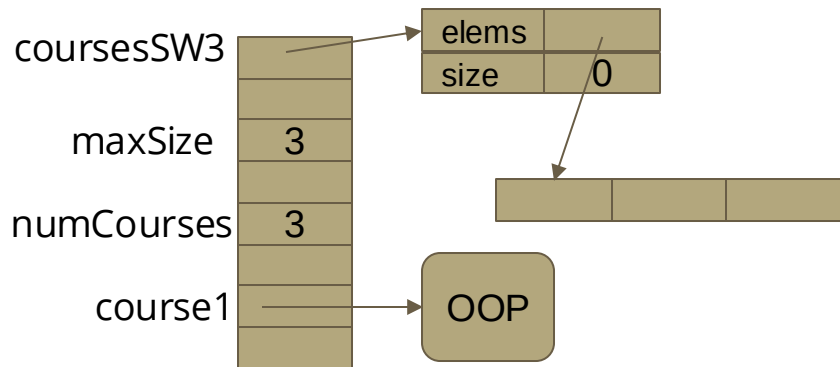


Quiz 5 - Parameter passing

Please provide your input using [this form](#)

Example 1

```
int numCourses = 3;  
Catalog coursesSW3 = new Catalog(numCourses);  
String course1 = "OOP";
```

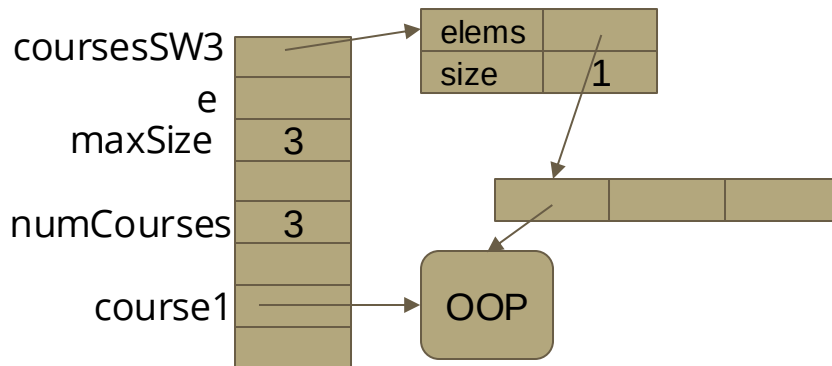


```
coursesSW3.addElem(course1);  
// point C
```

What is the value of **course1** at point C?

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
        // point A  
        e = null;  
        // point B  
    }  
}
```

What is the value of **e** at point A?



Example 2

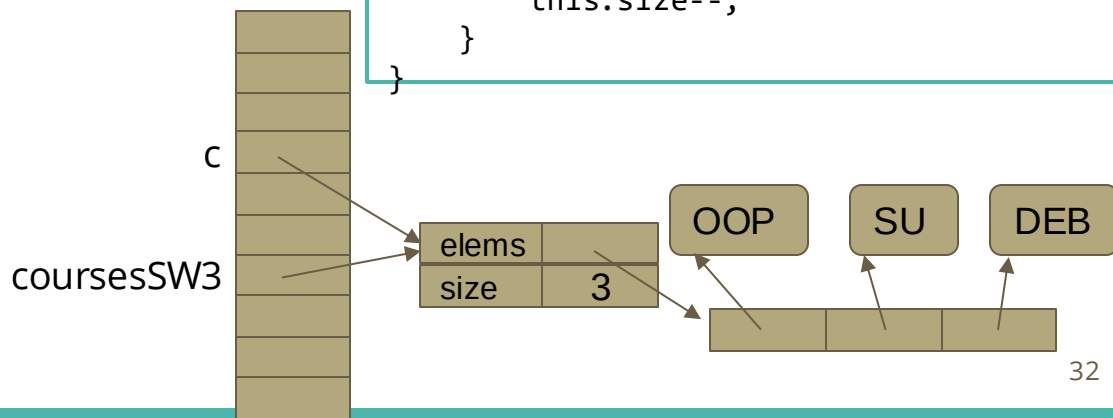
```
static void emptyCatalog(Catalog c) {  
    while (c.getSize() > 0) {  
        c.deleteLast();  
    }  
}
```

```
Catalog coursesSW3 = new Catalog(3);  
coursesSW3.addElem("OOP");  
coursesSW3.addElem("SU");  
coursesSW3.addElem("DEB");  
emptyCatalog(coursesSW3);
```

```
coursesSW3.getSize();
```

What is the value of
coursesSW3.getSize() ??

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new  
String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public int getSize() {  
        return size;  
    }  
    public void deleteLast() {  
        this.size--;  
    }  
}
```



Example 3

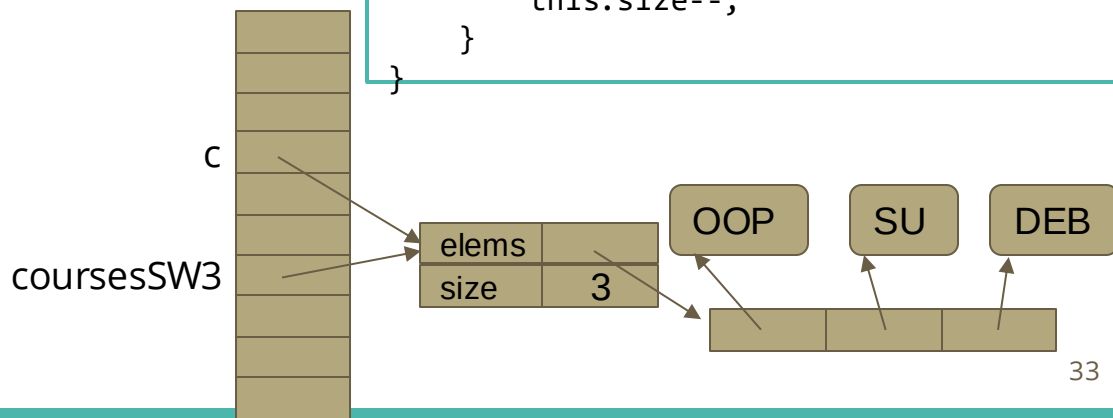
```
static void emptyCatalog(Catalog c) {  
    c = null;  
}
```

```
Catalog coursesSW3 = new Catalog(3);  
coursesSW3.addElem("OOP");  
coursesSW3.addElem("SU");  
coursesSW3.addElem("DEB");  
emptyCatalog(courseSW3);
```

```
coursesSW3.getSize();
```

What is the value of
coursesSW3.getSize() ??

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new  
String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public int getSize() {  
        return size;  
    }  
    public void deleteLast() {  
        this.size--;  
    }  
}
```



ArrayList<T> and Catalog<T> are **parameterized classes**

Generics

T is a **type variable**

- Allow for writing code that works on any type (universal/parametric **polymorphism**).
- **Formal type parameters** are placed inside angle brackets after the class declaration.
- **Actual parameters** must be a reference type.

```
import java.util.ArrayList;
class Catalog<T> {
    private ArrayList<T> elems;
    private String subject;
    private int size;
    public Catalog(String s) {
        this.subject = s;
        this.elems = new ArrayList<T>();
        this.size = 0;
    }
    public void addElem(T e) {
        this.elems.add(e);
        this.size++;
    }
    public int getSize() {
        return size;
    }
    public void deleteLast() {
        this.size--;
    }
}
```

```
Catalog<String> coursesSW3 = new Catalog<String>("Software, 3rd semester");
coursesSW3.addElem("OOP");
coursesSW3.addElem("SU");
coursesSW3.addElem("DEB");
```

Quiz 6 - Generics

Please provide your input using [this form](#)

Classes

- Templates used to create objects
- Define the **properties** (fields) and **behavior** (methods) of the objects
- Each object has a copy of all the instance fields (variables)
- Some fields and methods can be **static**
- **new** creates an instance of the class and constructors are used to initialize it
- **this** is used to refer to the current instance
- In Java, parameters are passed **by value** (primitive types) and **by reference value** (reference types) to methods and constructors

Appendix

Java Primitive Types

Table 4-3. List of Primitive Data Types, Their Size, Range, and Examples

Data Type	Size in Bits	Signed/Unsigned	Range	Example
byte	8	Signed	-2^7 to $+2^7 - 1$	-2, 8, 10
short	16	Signed	-2^{15} to $+2^{15} - 1$	-2, 8, 10
int	32	Signed	-2^{31} to $+2^{31} - 1$	1990, -90, 23
long	64	Signed	-2^{63} to $+2^{63} - 1$	1990L, -90L, 23L
char	16	Unsigned	0 to 65535	'A', '8', '\u0000'
float	32	Signed	-3.4×10^{38} to $+3.4 \times 10^{38}$	12.89F, -89.78F
double	64	Signed	-1.7×10^{308} to $+1.7 \times 10^{308}$	12.78, -78.89
boolean	Unspecified	N/A	true and false	true, false

Some Constants

Table 4-5. Constants Defined in the Float Class

Float Constants	Meaning
Float.POSITIVE_INFINITY	Positive infinity of type float.
Float.NEGATIVE_INFINITY	Negative infinity of type float.
Float.NaN	Not-a-Number of type float.
Float.MAX_VALUE	The largest positive value that can be represented in a float variable. This is equal to 3.4×10^{38} (approx.).
Float.MIN_VALUE	The smallest positive value greater than zero that can be represented in a float variable. This is equal to 1.4×10^{-45} .

Table 4-6. Constants in the Double Class

Double Constants	Meaning
Double.POSITIVE_INFINITY	Positive infinity of type double.
Double.NEGATIVE_INFINITY	Negative infinity of type double.
Double.NaN	Not-a-Number of type double.
Double.MAX_VALUE	The largest positive value that can be represented in a double variable. This is equal to 1.7×10^{308} (approx.).
Double.MIN_VALUE	The smallest positive value greater than zero that can be represented in a double variable. This is equal to 4.9×10^{-324} .

Module `java.base`

Package `java.lang`

Class Integer

`java.lang.Object`
`java.lang.Number`
`java.lang.Integer`

All Implemented Interfaces:

`Serializable`, `Comparable<Integer>`, `Constable`, `ConstantDesc`

`public final class Integer`

`extends Number`

`implements Comparable<Integer>`, `Constable`, `ConstantDesc`

Fields			
Modifier and Type	Field	Description	
<code>static final int</code>	<code>BYTES</code>	The number of bytes used to represent an <code>int</code> value in two's complement binary form.	
<code>static final int</code>	<code>MAX_VALUE</code>	A constant holding the maximum value an <code>int</code> can have, $2^{31}-1$.	
<code>static final int</code>	<code>MIN_VALUE</code>	A constant holding the minimum value an <code>int</code> can have, -2^{31} .	

Arithmetic Operators

Table 5-1. List of Arithmetic Operators in Java

Operators	Description	Type	Usage	Result
+	Addition	Binary	2 + 5	7
-	Subtraction	Binary	5 - 2	3
+	Unary plus	Unary	+5	Positive of five. Same as 5.
-	Unary minus	Unary	-5	Negative of five.
*	Multiplication	Binary	5 * 3	15
/	Division	Binary	5 / 2	2
			6 / 2	3
			5.0 / 2.0	2.5
			6.0 / 2.0	3.0
%	Modulus	Binary	5 % 3	2
++	Increment	Unary	num++	Evaluates to the value of num. Increment num by 1.
--	Decrement	Unary	num--	Evaluates to the value of num. Decrement num by 1.
+=	Arithmetic compound assignment	Binary	num += 5	Adds 5 to the value of num and assigns the result to num. If num is 10, the new value of num will be 15.
-=	Arithmetic compound assignment	Binary	num -= 3	Subtracts 3 from the value of num and assigns the result to num. If num is 10, the new value of num will be 7.
*=	Arithmetic compound assignment	Binary	num *= 15	Multiplies 15 by the value of num and assigns the result to num. If num is 10, the new value of num will be 150.
/=	Arithmetic compound assignment	Binary	num /= 5	Divides the value of num by 5 and assigns the result to num. If num is 10, the new value of num will be 2.
%=	Arithmetic compound assignment	Binary	num %= 5	Calculates the remainder of num divided by 5 and assigns the result to num. If num is 12, the new value of num will be 2.

Logical Operators

Table 5-4. List of Boolean Logical Operators

Operators	Meaning	Type	Usage	Result
!	Logical NOT	Unary	!true	false
&&	Short-circuit AND	Binary	true && true	true
&	Logical AND	Binary	true & true	true
	Short-circuit OR	Binary	true false	true
	Logical OR	Binary	true false	true
^	Logical XOR (Exclusive OR)	Binary	true ^ true	false
&=	AND assignment	Binary	test &= true	
=	OR assignment	Binary	test = true	
^=	XOR assignment	Binary	test ^= true	

Table 5-5. Compound Logical Assignment Operators and Their Equivalents

Expression	Is Equivalent To
operand1 &= operand2	operand1 = operand1 & operand2
operand1 = operand2	operand1 = operand1 operand2
operand1 ^= operand2	operand1 = operand1 ^ operand2

Java Operators and Their Precedence

Table 5-8. Java Operators and Their Precedence

Level	Operator Symbol	Action Performed			
1	++	Pre- or post-increment			
	--	Pre- or post-decrement	6	==	Equal in value
	+, -	Unary plus, unary minus		!=	Not equal to
	~	Bitwise complement	7	&	Bitwise AND
	!	Logical NOT		&	Logical AND
	(type)	Cast	8	^	Bitwise XOR
2	*, /, %	Multiplication, division, remainder		^	Logical XOR
			9		Bitwise OR
3	+, -	Addition, subtraction			Logical OR
	+	String concatenation	10	&&	Logical short-circuit AND
4	<<	Left shift	11		Logical short-circuit OR
	>>	Signed right shift	12	? :	Ternary operator
	>>>	Unsigned right shift	13	=	Assignment
5	<	Less than		+=, -=, *=, /=, %=,	Compound assignment
	<=	Less than or equal		<<=, >>=, >>>=,	
	>	Greater than		&=, =, ^=	
	>=	Greater than or equal			
	instanceof	Type comparison			

== and primitive types I

```
int i = 0;
```

```
String s = "0";
```

```
boolean b = (i == s);
```

```
jshell> int i = 0;  
i ==> 0  
  
jshell> String s = "0";  
s ==> "0"  
  
jshell> boolean b = (i == s);  
|   Error:  
|   bad operand types for binary operator '=='  
|       first type:  int  
|       second type: java.lang.String  
|   boolean b = (i == s);  
|                   ^-----^  
  
jshell>
```

What is the value of b after executing these commands?

Both operands must be either reference or primitive

== and primitive types II

```
int i = 5;
```

```
double d = 5d;
```

```
boolean b = (i == d);
```

```
jshell> int i = 5;
```

```
i ==> 5
```

```
jshell> double d = 5d;
```

```
d ==> 5.0
```

```
jshell> boolean b = (i == d);
```

```
b ==> true
```

What is the value of b after executing these commands?

Java converts the int to double as needed

Variable arity/Var-args Methods

- **type ...** denotes a variable arity parameter, i.e., zero or more parameters of type **type**
- There can be **at most one** variable arity parameter and must be the last one

```
Catalog coursesSW3 = new Catalog(3);  
coursesSW3.addElem("OOP", "SU", "DEB");
```

The compiler translates String... to String[]

```
public class Catalog {  
    private String[] elems = null;  
    private int size = 0;  
    private static int count = 0;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        count++;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public void addElem(String... es) {  
        for (String e : es) {  
            this.addElem(e);  
        }  
    }  
    public static int getCount() {  
        return Catalog.count;  
    }  
}
```